Progetto semestre - SUPSI DTI

- **Data**: 2024-22-04
- **Versione**: 1.0
- **Collaboratori**: Carlo Pezzotti, Kushtrim Rushi

Abstract

The project aimed to develop an inclusive communication application, leveraging pictograms and personalized features to facilitate interactions for individuals with language difficulties or cognitive disabilities. Key features included a wide array of pictograms sourced from Araasac, customizable font sizes and colors, and speech-to-text and text-to-speech functionalities. The management of chats was facilitated through integration with Telegram. Through React Native for the front-end and Flask for the back-end, the app provided an intuitive interface and efficient communication channels. Despite the complexities involved, the project team found great satisfaction in overcoming challenges and innovating to create a meaningful solution for its target users

Introduzione

Descrizione dell'idea:
Pictochat è un'applicazione di comunicazione progettata per facilitare la comunicazione tra ragazzi che possono avere difficoltà di linguaggio o disabilità cognitive. L'app offre una vasta gamma di pittogrammi e immagini tratti da Araasac, insieme alla possibilità di aggiungere immagini personalizzate e loghi.

Obiettivo:
L'obiettivo principale di Pictochat è rendere la comunicazione accessibile e inclusiva, permettendo ai ragazzi di esprimersi in modo chiaro tra di loro o ai genitori di comprendere meglio i loro bisogni e desideri.

Caratteristiche principali:

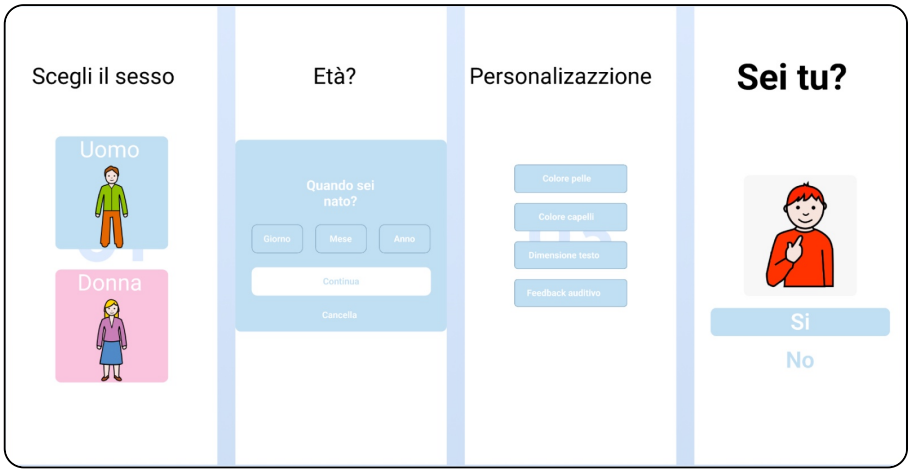
1. **Comunicazione tramite pittogrammi:**
    - Ampia raccolta di pittogrammi tratti da Araasac per rappresentare persone, luoghi e azioni.
    - Filtraggio diretto delle immagini in base alle scelte del soggetto, per una ricerca più efficiente. (Da implementare)
2. **Personalizzazione e accessibilità:**
    - Selezione del font size e del colore dei pittogrammi per una migliore leggibilità e accessibilità, con opzioni adatte anche ai dislessici.
3. **Funzionalità di comunicazione:**
    - **Speech to text:** Trascrizione automatica dei messaggi vocali in testo per una migliore comprensione.
    - **Text to speech:** Lettura automatica del testo inserito per una comunicazione bidirezionale.
    - Creazione di preset di frasi standard personalizzabili per consentire una comunicazione più rapida e efficiente.
4. **Interfaccia intuitiva:**
    - Home page personalizzabile con immagini delle persone contattate e opzioni di espressione emotiva.
    - Semplificazione dell'interfaccia per evitare sovraccarico e frustrazione, con un design intuitivo e user-friendly.
5. **Integrazione con IA:** (Da implementare)
    - Integrazione con un sistema di intelligenza artificiale per convertire le immagini in testo e viceversa, per una comunicazione più fluida e naturale.
    - Suggerimenti automatici di pittogrammi e frasi in base al contesto e alle esigenze del soggetto.
        - Per esempio, se il soggetto seleziona il verbo "mangiare", l'app suggerirà automaticamente i pittogrammi relativi a cibo e bevande.

Target di utenza:
Il target principale sono i ragazzi con difficoltà di linguaggio o disabilità cognitive, insieme ai ragazzi stessi che utilizzano l'app per comunicare con i loro amici, genitori, tutor o operatori scolastici.

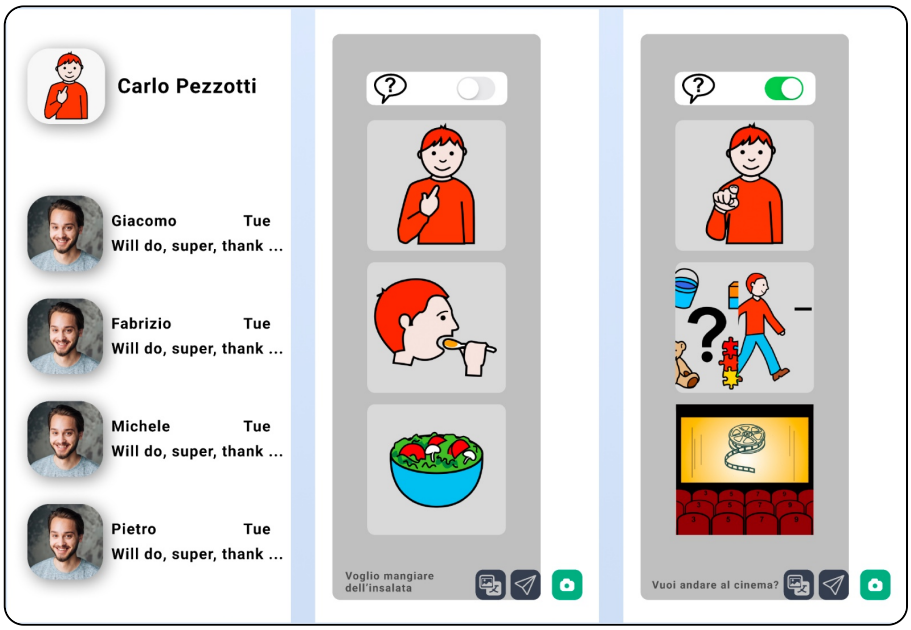
Design

Mockup

Il nostro mockup offre uno sguardo preliminare alla piattaforma della chat in fase di sviluppo. Illustra l'interfaccia di base della chat, evidenziando i messaggi, i controlli di accessibilità e le opzioni di personalizzazione.



Nella fase iniziale dell'app, si richiede agli utenti di fornire informazioni fondamentali per personalizzare la loro esperienza. Queste informazioni includono il genere, l'anno di nascita (per calcolare l'età) e il colore dei capelli e della pelle. Questi dettagli consentono al sistema di creare un pittogramma rappresentativo dell'utente, dove possibile, migliorando così la comunicazione visiva e facilitando l'interazione tra gli utenti.

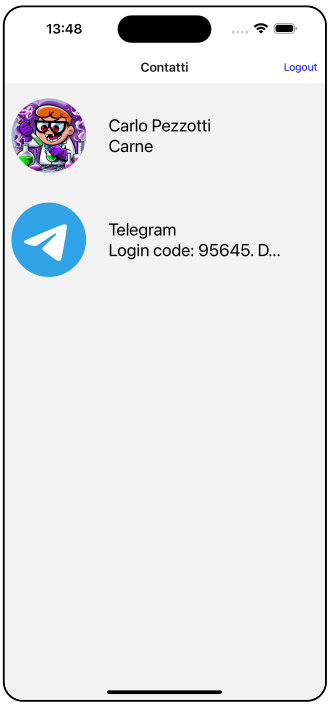


L'immagine illustra l'interfaccia della lista delle chat nell'applicazione, mostrando le conversazioni attive degli utenti. Successivamente, viene presentata un'esemplificazione del processo di composizione di una frase mediante l'utilizzo di pittogrammi. Gli utenti possono selezionare tre pittogrammi, rappresentanti il soggetto, il verbo e il complemento oggetto, per comporre una frase. Utilizzando un'intelligenza artificiale, viene generata una frase completa che permette agli utenti di inviare messaggi senza la necessità di digitare manualmente sulla tastiera.

Final Product

Questa sezione offre una panoramica concisa del prodotto completo, mostrando l'interfaccia utente finale e le principali funzionalità.

Il processo di accesso avviene tramite il numero di telefono, accompagnato dal prefisso corretto (`+41` o `0041`). Dopo aver inserito il numero, l'utente riceverà un codice di verifica tramite Telegram. Questo codice dovrà essere inserito nell'applicazione per completare la procedura di accesso e accedere al proprio account.



Nell'immagine sopra è visibile l'intera lista delle chat attive, con le immagini profilo degli utenti prese da Telegram.

Nella sezione delle risposte/frasi predefinite, gli utenti possono selezionare alcune frasi preimpostate, senza la necessità di digitare manualmente sulla tastiera. Queste frasi sono utili per domande di uso quotidiano, offrendo un modo rapido e conveniente per interagire all'interno dell'applicazione.

Nell'immagine mostrata, viene illustrato come gli utenti della nostra applicazione possono rispondere alle domande poste da altri utenti. Il sistema estrae la parola chiave più rilevante dalla domanda e genera un pittogramma corrispondente, se disponibile, che viene visualizzato insieme al testo della domanda. Gli utenti possono quindi scegliere tra diverse opzioni rappresentate sotto forma di pittogrammi, scorrendo lateralmente. Una volta selezionata un'opzione, questa viene automaticamente inserita nella chat sotto forma di testo, insieme al relativo pittogramma.

Architettura Generale

Front-end (React Native)

Il front-end dell'applicazione è sviluppato con React Native, un framework JavaScript per la creazione di app mobili multi-piattaforma. Questa parte gestisce l'interfaccia utente, garantendo una presentazione fluida e intuitiva delle funzionalità dell'applicazione.

Back-end (Python)

Il back-end è realizzato in Python utilizzando Flask, un framework leggero per la creazione di applicazioni web. Questa componente è responsabile della gestione delle richieste del client e della comunicazione con l'API di Telegram per la gestione dei messaggi.

Il back-end si avvale di diverse librerie esterne come **aiohttp**, **requests** e **Telethon** per facilitare la gestione delle richieste e delle sessioni con l'API di Telegram, assicurando un'interazione efficiente e affidabile con la piattaforma di messaggistica.

Questa architettura permette una sinergia efficace tra il front-end sviluppato in React Native e il back-end implementato in Python tramite Flask, garantendo un'esperienza utente coerente e una gestione ottimale delle comunicazioni tramite Telegram.

Immagazzinamento dei dati

Per garantire la persistenza delle informazioni e agevolare il ripristino delle conversazioni in caso di chiusura dell'applicazione o di riavvio del dispositivo, le sessioni degli utenti vengono memorizzate in un file JSON locale.

Ecco un esempio della struttura del file JSON:

```
{
    "[token_utente]": "[sessione_utente]"
}
```

In questo modo, ogni token utente è associato alla propria sessione, consentendo un facile accesso e recupero delle informazioni necessarie per l'interazione con l'API di Telegram. Questo approccio garantisce una gestione efficiente delle sessioni degli utenti e contribuisce a una migliore esperienza d'uso dell'applicazione.

Tecnologie Utilizzate

Nel processo di sviluppo dell'applicazione, sono state impiegate diverse tecnologie sia per il front-end che per il back-end al fine di garantire funzionalità avanzate e una user experience ottimale.

Front-end

- **React Native**: Framework JavaScript per lo sviluppo di applicazioni mobili cross-platform, che consente di utilizzare lo stesso codice sorgente per Android e iOS.
- **@react-navigation**: Libreria per la gestione della navigazione all'interno dell'applicazione, offrendo diverse soluzioni per la navigazione tra schermate e stack di navigazione.
- **@ant-design**: Raccolta di componenti UI React Native, che fornisce una vasta gamma di componenti pronti all'uso per la creazione di interfacce utente moderne e intuitive.
- **Expo**: Piattaforma che semplifica lo sviluppo di app React Native, fornendo strumenti per la creazione, il test e la distribuzione delle applicazioni.
- **AsyncStorage**: API React Native per la memorizzazione locale dei dati, utilizzata per salvare le sessioni degli utenti e garantire la persistenza delle informazioni.

Back-end

- **Flask**: Micro-framework Python per lo sviluppo di applicazioni web e back-end, noto per la sua semplicità e flessibilità.
- **aiohttp**: Libreria Python per la creazione di server web asincroni, che offre un'elevata scalabilità e prestazioni ottimizzate per applicazioni web ad alta intensità di traffico.
- **Telethon**: Libreria Python per l'interfacciamento con l'API di Telegram, che facilita l'invio e la ricezione di messaggi tramite la piattaforma di messaggistica. Il fulcro dell'applicazione è la comunicazione con l'API di Telegram, che consente agli utenti di inviare e ricevere messaggi tramite l'applicazione. Questa libreria si occupa di gestire il protocollo MTProto e di fornire un'interfaccia semplice e intuitiva per l'interazione con l'API.
- **requests**: Libreria Python per l'invio di richieste HTTP, utilizzata per comunicare con servizi esterni e integrare funzionalità aggiuntive nell'applicazione.

Implementazione Python (Flask)

```python
@app.route('/me', methods=['GET'])
async def get_me():
    token = request.headers.get('token')
    result = await get_telegram_personal_info(token)
    return result.to_json()
```

In questa implementazione con Flask, viene definito un endpoint `/me` che gestisce le richieste GET. L'endpoint estrae il token dall'header della richiesta e utilizza questa informazione per recuperare le informazioni personali dell'utente tramite la funzione `get_telegram_personal_info()`. Infine, le informazioni vengono convertite in formato JSON e restituite come risposta.

Implementazione TypeScript (Front-end)

```typescript
static getMe: () => Promise<User> = async () => {
    const res = await fetch(`${this.url}/me`, {
        headers: {
            "Accept": "application/json",
            "token": this.token
        },
        method: "GET"
    });
    const data = await res.json();
    return data;
}
```

Nel codice TypeScript, viene definito un metodo statico `getMe()` all'interno di una classe. Questo metodo utilizza la funzione `fetch()` per inviare una richiesta GET all'endpoint `/me` del server. Gli header della richiesta includono le specifiche per accettare dati JSON e il token dell'utente per l'autenticazione. Una volta ricevuta la risposta dal server, i dati vengono convertiti in formato JSON e restituiti come risultato della funzione. Per ogni endpoint, vengono definite funzioni simili per gestire le richieste HTTP e comunicare con il server. Questo approccio consente di mantenere un codice pulito e ben strutturato, facilitando la gestione delle chiamate API e garantendo una comunicazione efficiente tra il front-end e il back-end dell'applicazione. Tra gli endpoint implementati, vi sono:

- **/me**: Restituisce le informazioni personali dell'utente.
- **/contacts**: Ottiene i contatti dell'utente.
- **/contacts/**: Ottiene le informazioni di un contatto specifico tramite il suo identificatore.
- **/messages//**: Ottiene i messaggi di una conversazione fino a un limite specificato.
- **/messages/**: Ottiene tutti i messaggi di una conversazione.
- **/messages/send/**: Invia un messaggio a un determinato destinatario.
- **/messages/send/image/**: Invia un'immagine con una didascalia a un destinatario specifico.
- **/login/<phone_number>**: Avvia il processo di login richiedendo il numero di telefono.
- **/login/confirm**: Conferma il processo di login con il numero di telefono e il codice di verifica.

Struttura

Front-end

- **App.tsx**: Componente principale dell'applicazione, che gestisce la navigazione tra le diverse schermate e definisce il layout generale dell'interfaccia utente.
- **screens/**: Cartella contenente le schermate dell'applicazione, suddivise in base alle funzionalità e ai flussi di navigazione.
    - **IndexScreen.tsx**: Schermata principale dell'applicazione, che mostra, nel caso nuovi nell'applicazione, la schermata di login, altrimenti reindirizza alla schermata dei contatti.
    - **ContactScreen.tsx**: Schermata che visualizza i contatti dell'utente e consente di avviare una conversazione con un contatto selezionato.
    - **ChatScreen.tsx**: Schermata di chat, che mostra i messaggi scambiati con un contatto specifico e consente di inviare nuovi messaggi.
- **lib/**: Cartella contenente le funzioni di utilità e le chiamate API per la comunicazione con il server back-end.
    - **storege.ts**: Funzioni per la gestione della memorizzazione locale dei dati.
    - **AAC.ts**: Classe che gestisce le chiamate API per l'interazione con il server ARASAAC.
    - **TgAPI.ts**: Classe che gestisce le chiamate API per l'interazione con il server python.
    - **Utils.ts**: Funzioni di utilità per la gestione dei dati e delle operazioni comuni.

Back-end

Il backend, in quanto basato interamente su server telegram si struttura in modo molto semplice, con un unico file principale: Il file index.py contiene le seguenti funzioni principali:

- **create_token()**: Questa funzione genera un token casuale di 32 caratteri utilizzato per identificare univocamente una sessione utente.
- **dialog_to_json(dialog: Dialog)**: Questa funzione converte un oggetto Dialog di Telegram in un formato JSON compatibile, estratte le informazioni essenziali come il nome del dialogo, la data e i messaggi.
- **gen_telegram_client(token)**: Questa funzione genera un cliente Telegram utilizzando il token fornito, utilizzato per la comunicazione con l'API di Telegram.
- **get_telegram_client(token)**: Questa funzione restituisce un cliente Telegram già connesso se il token è già stato utilizzato per la connessione in precedenza, altrimenti crea un nuovo client e lo restituisce.
- **ask_for_login(phone_number)**: Questa funzione avvia il processo di login tramite l'invio del codice di verifica al numero di telefono specificato.
- **confirm(token, phone_number, code, phone_hash)**: Questa funzione conferma il processo di login utilizzando il token, il numero di telefono, il codice di verifica e l'hash del telefono.

Le restanti funzioni gestiscono le operazioni di recupero delle informazioni da Telegram, come ottenere le informazioni personali dell'utente, i contatti, i messaggi e l'invio di nuovi messaggi o immagini. Tutte queste funzioni sfruttano il client Telegram ottenuto per comunicare con l'API di Telegram e svolgere le operazioni richieste.

Autenticazione e Autorizzazione

Fortunatamente la autenticazione è gestita interamente da telegram e non è necessario implementare un sistema di autenticazione personalizzato. Per quanto riguarda l'autorizzazione, l'applicazione utilizza un sistema di token per identificare gli utenti e consentire l'accesso alle risorse protette. Ogni richiesta al server include un token univoco associato all'utente, che viene verificato per garantire l'autorizzazione alle risorse richieste.

Il token viene generato al momento del login e memorizzato localmente sul dispositivo dell'utente. Questo token viene utilizzato per autenticare le richieste al server e garantire che solo gli utenti autorizzati possano accedere alle risorse protette.

Esempio di token: `CRlq3zAgbmhq2evjj4CE58upjh5zv8UW`

Risorse Utili

Araasac: https://arasaac.org Telegram API: https://docs.telethon.dev React Native: https://reactnative.dev MTProto: https://core.telegram.org/mtproto Flask: https://flask.palletsprojects.com aiohttp: https://docs.aiohttp.org requests: https://docs.python-requests.org

Conclusioni

Il progetto ha richiesto un'immersione completa nell'esperienza degli utenti con difficoltà di linguaggio e cognitive per sviluppare un'app efficace e significativa. Abbiamo affrontato sfide significative per rendere la comunicazione più accessibile e inclusiva, trovando soluzioni innovative lungo il percorso. Nonostante le difficoltà, il lavoro è stato gratificante, migliorando costantemente l'applicazione per gli utenti finali. Questa esperienza ci ha arricchito sia tecnicamente che emotivamente, spingendoci a pensare in modo innovativo e centrato sull'utente, confermando l'importanza dell'inclusione nella progettazione software. Guardiamo al futuro con orgoglio e entusiasmo, consapevoli del positivo impatto che il nostro lavoro può avere sulla vita delle persone.

Sviluppi futuri

- Aggiunta opzione "Ascolta" nella lista delle frasi predefinite con emoji distintiva.
- Sfondo della chat scuro e pittogrammi chiari per maggiore leggibilità.
- Implementazione di foto personalizzate per l'uso dei pittogrammi.
- Introduzione di conferma di invio al click sui pittogrammi.
- Aggiunta di un'opzione rapida per disattivare il text-to-speech.
- Associazione di un punto di domanda alle domande prive di pittogramma.
- Implementazione della funzionalità pittogrammi-to-text (soggetto, verbo, oggetto) come mostrato nel mockup.
- Integrazione con Intelligenza Artificiale per la generazione di immagini.